

CS 550 Operating Systems

Spring 2026

Paging: faster translation (TLBs)

Introduction

- High performance overhead of paging
 - Slow: extra memory access for each virtual address
 - large amount of mapping information (in memory)
- Solution: hardware support - **translation-lookaside buffer (TLB)**
 - Part of the CPU chip's MMU
 - Essentially a hardware cache of popular VPN-to-PFN translations
 - Upon each virtual memory reference, hardware first checks TLB to see if the desired translation is held therein; if so, the translation is performed (quickly) without having to consult the page table (which has all translations).

VA to PA translation (with TLB)

```
1 VPN = (VirtualAddress & VPN_MASK) >> SHIFT  Extract VPN from VA
2 (Success, TlbEntry) = TLB Lookup(VPN)  Check if TLB holds the translation for the VA
3 if (Success == True)  TLB hit
4     if (CanAccess(TlbEntry.ProtectBits) == True)  Can access the translation
5         Offset = VirtualAddress & OFFSET_MASK  Get the offset from VA
6         PhysAddr = (TlbEntry.PFN << SHIFT) | Offset  Form the PA
7         AccessMemory(PhysAddr)  Access the memory location
8     else
9         RaiseException(PROTECTION_FAULT)  No privilege to access the memory
10 else  // TLB Miss
11     PTEAddr = PTBR + (VPN * sizeof(PTE))  Get the address of the PTE
12     PTE = AccessMemory(PTEAddr)  Get the PTE from memory
13     if (PTE.Valid == False)  Cannot access the translation b/c it is invalid
14         RaiseException(SEGMENTATION_FAULT)
15     else if (CanAccess(PTE.ProtectBits) == False)  Cannot access the
16         RaiseException(PROTECTION_FAULT)  translation no privilege
17     else  Can access the translation
18         TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
19         RetryInstruction()
```

Example: access array

- 8-bit virtual address space and 16-byte pages
 - Address space range? 256 bytes
 - # of VPN bits? 4
 - # of offset? 4
- 10 4-byte integers starting at VA 100

```
int sum = 0;
for (i = 0; i < 10; i++) {
    sum += a[i];
}
```

a[0] – miss

a[1] – hit

a[2] – hit

a[3] – miss a[4] – hit a[5] – hit a[6] – hit

a[7] – miss a[8] – hit a[9] – hit

TLB hit rate: 7/10 = 70%

	Offset				
	00	04	08	12	16
VPN = 00					
VPN = 01					
VPN = 02					
VPN = 03					
VPN = 04					
VPN = 05					
VPN = 06		a[0]	a[1]	a[2]	
VPN = 07	a[3]	a[4]	a[5]	a[6]	
VPN = 08	a[7]	a[8]	a[9]		
VPN = 09					
VPN = 10					
VPN = 11					
VPN = 12					
VPN = 13					
VPN = 14					
VPN = 15					

- TLB improves performance due to **spatial locality**.
 - The elements of the array are packed tightly into pages, and thus **only the first access to an element on a page** yields a TLB miss.
- Impact of page size
 - If the page size is twice as big (i.e., 32 bytes), what is the TLB hit rate?

	Offset				
	00	04	08	12	16
VPN = 00					
VPN = 01					
VPN = 02					
VPN = 03					
VPN = 04					
VPN = 05					
VPN = 06		a[0]	a[1]	a[2]	
VPN = 07	a[3]	a[4]	a[5]	a[6]	
VPN = 08	a[7]	a[8]	a[9]		
VPN = 09					
VPN = 10					
VPN = 11					
VPN = 12					
VPN = 13					
VPN = 14					
VPN = 15					

- TLB improves performance due to **spatial locality**.
 - The elements of the array are packed tightly into pages, and thus **only the first access to an element on a page** yields a TLB miss.
- Impact of page size
 - If the page size is twice as big (i.e., 32 bytes), what is the TLP hit rate? **80%**
 - Typical page size is 4KB. Therefore these types of dense, array-based accesses achieve excellent TLB performance.
- If the program, soon after this loop completes, accesses the array again, what would be the hit rate? **100%**
 - The TLB hit rate would be high because of **temporal locality**.

	Offset				
	00	04	08	12	16
VPN = 00					
VPN = 01					
VPN = 02					
VPN = 03					
VPN = 04					
VPN = 05					
VPN = 06		a[0]	a[1]	a[2]	
VPN = 07	a[3]	a[4]	a[5]	a[6]	
VPN = 08	a[7]	a[8]	a[9]		
VPN = 09					
VPN = 10					
VPN = 11					
VPN = 12					
VPN = 13					
VPN = 14					
VPN = 15					

Caching and Locality

- Caching is one of the most fundamental performance techniques in computer systems to make common-case faster
- Idea behind caching is to take advantage of **locality** in instruction and data references
- **Temporal** locality: an instruction or data item that has been recently accessed will likely be re-accessed soon in the future (*e.g.*, instructions in a loop)
- **Spatial** locality: if program accesses memory x , it will likely soon access memory near x

Who handles TLB misses

- Two possible answers: hardware or software (the OS).
- Hardware
 - Commonly used in CISC (e.g., Intel x86)
 - H/W handles TLB misses entirely: it need to know where the page tables are located in the memory (page-table base register), and the exact format.
 - On a TLB miss, the hardware would “walk” the page table, find the correct page-table entry and extract the desired translation, update the TLB with the translation, and retry the instruction.

- Software
 - Software-managed TLB Commonly used in RISC (e.g., MIPS, ARM).
 - On a TLB miss, the hardware raises an exception, which pauses the current instruction stream, raises the privilege level to kernel mode, and jumps to a trap handler.
 - The trap handler looks up the translation in the page table, use special “privileged” instructions to update the TLB, and return from the trap.
 - The return-from-trap instruction on TLB miss handling
 - System call trap: execute the next instruction
 - TLB miss trap: execute the instruction that caused the trap

TLB contents

- 32, 64, or 128 entries
- Fully associative: any given translation can be anywhere in TLB, and hardware will search the entire TLB in parallel to find the desired translation
- An entry looks like: VPN | PFN | other bits
 - Valid bit
 - TLB valid bit $\neq$ page table valid bit $\rightarrow$ in page table, when a PTE is marked invalid, it means that the page has not been allocated by the process; a TLB valid bit refers to whether a TLB entry has a valid translation within it
 - Protection bit: R/W/X
 - Dirty bit
 - Address space identifier
 - ...

Context switch

- TLB contains virtual-to-physical page translations that are **only valid for the currently running process**; these translations are not meaningful for other processes.

	VPN	PFN	valid	prot
P1 →	10	100	1	rwX
	—	—	0	—
P2 →	10	170	1	rwX
	—	—	0	—

What to do to avoid this from happening?

Context switch

- Solution 1: flush TLB on context switches
 - On a software-based system, this can be accomplished with an explicit (and privileged) hardware instruction.
 - With a hardware-managed TLB, the flush could be done when the page-table base register is changed.
 - In either case, the flush operation simply sets all valid bits to 0, essentially clearing the contents of the TLB.

Context switch

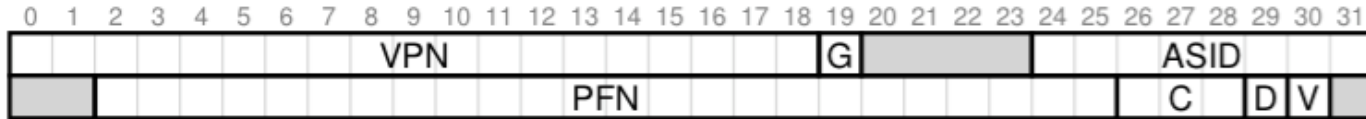
- Solution 2: add hardware support to enable sharing of the TLB across context switches.
 - Address space identifier (ASID).
 - ARM, MIPS, ...
 - Similar to PID → to identify individual process
 - Usually 8 bits

VPN	PFN	valid	prot	ASID
10	100	1	rwX	1
—	—	0	—	—
10	170	1	rwX	2
—	—	0	—	—

Replacement policy

- When we are installing a new entry in the TLB, we have to replace an old one, and thus the question: which one to replace?
 - Common approach: LRU
 - Corner case: a program loops over $n + 1$ pages with a TLB of size n
 - Another common approach: random
 - More on this when discussing page replacement policy

A real TLB entry



- MIPS R4000 (software-managed TLBs)
- 64 bit TLB entry
- VPN: Virtual Page Number
- PFN: Physical Frame Number (24 bit PFN → support up to 64 GB physical memory (2^{24} 4K pages))
- Global bit (G): indicate if a page is globally shared among processes.
- ASID (8 bit)
- Coherence bits (C): how a page is cached by hardware
- Valid bit (v)
- Dirty bit (d)